Busca Local e Algoritmo Genético para o Sequenciamento de Manutenção de Ativos: uma Análise Comparativa

Rafael Campello Soares, Arthur Rafael Silva Teixeira
Roberto Júnior Cassimiro Campolina, Lucas Araújo Campos Szuster
Universidade Federal de Minas Gerais (UFMG) – Departamento de Engenharia Elétrica
Disciplina: Fundamentos de Inteligência Artificial – Trabalho Prático 2
Belo Horizonte, MG, Brasil

Resumo Este trabalho compara três métodos de otimização — *greedy descent* com *random restart*, *simulated annealing* e algoritmo genético — aplicados ao sequenciamento de manutenção preventiva de ativos de uma concessionária de energia, formulado como a minimização do atraso ponderado total sobre permutações. Os métodos são avaliados em três instâncias ($n = 12$, $n = 50$ e $n = 150$) sob um orçamento idêntico de avaliações da função objetivo e com execuções independentes. **[PREENCHER: resumir os principais resultados quantitativos e a conclusão central]**

Palavras-chave busca local; *simulated annealing*; algoritmo genético; otimização combinatória; sequenciamento; atraso ponderado total.

1 Introdução

Uma concessionária de distribuição de energia precisa executar a manutenção preventiva de n ativos (religadores, transformadores de distribuição e chaves seccionadoras) com uma única equipe, que parte de uma base e atende um ativo por vez. Cada ativo possui posição, duração de serviço, prazo regulatório e um peso de criticidade que representa o custo por hora de atraso. A ordem de visita determina tanto o tempo gasto em deslocamentos quanto os atrasos incorridos, e o objetivo é encontrar a ordem que minimiza o atraso ponderado total.

O espaço de busca possui $n!$ soluções; para $n = 150$, esse número tem mais de 260 algarismos, o que inviabiliza qualquer enumeração. Trata-se de um problema de otimização combinatória sem restrições rígidas: toda permutação é viável e há apenas uma função de custo a minimizar. Dois critérios gulosos antagônicos — visitar o ativo mais próximo ou o mais urgente — não produzem, isoladamente, boas soluções, o que torna o problema adequado para comparar métodos de busca mais elaborados.

O objetivo do trabalho é comparar, em um problema prático de engenharia, heurísticas de busca local e uma metaheurística populacional, observando na prática o equilíbrio entre *exploração* (visitar regiões diferentes do espaço de busca) e *intensificação* (refinar soluções em uma região promissora). Os três algoritmos são:

- **(A)** *greedy descent* com *random restart*;
- **(B)** *simulated annealing*;
- **(C)** algoritmo genético (*genetic algorithm*, GA).

A comparação segue um protocolo único: o recurso contabilizado é o número de avaliações da função h, e todos os algoritmos recebem o mesmo orçamento por instância (Seção 4). A pergunta central, a ser respondida com dados na Seção 5, é qual a relação entre cada algoritmo e a estrutura do problema — em particular, se a recombinação do GA consegue combinar bons trechos de sequência de dois pais.

Organização. A Seção 2 apresenta a modelagem formal. A Seção 3 descreve a arquitetura do código e os três algoritmos. A Seção 4 detalha a metodologia experimental. A Seção 5 apresenta os resultados e a análise comparativa, e a Seção 6 traz as conclusões.

2 Modelagem do Problema

2.1 Dados da instância

Seja $\mathcal{A} = \{1, \ldots, n\}$ o conjunto de ativos e 0 o índice da base. Para cada ativo $j \in \mathcal{A}$ são dados a posição (x_j, y_j), a duração do serviço p_j (horas), o prazo d_j (hora-limite de conclusão) e o peso de criticidade w_j (custo por hora de atraso). A base tem posição (x_0, y_0) e $p_0 = 0$. O tempo de deslocamento entre dois pontos i e j é

$$t_{ij} = \operatorname{round}\left(\sqrt{(x_i - x_j)^2 + (y_i - y_j)^2}\right), \qquad (1)$$

com arredondamento para o inteiro mais próximo. Esse arredondamento é necessário para que os valores de h sejam comparáveis entre grupos. A equipe não retorna à base ao final.

2.2 Espaço de estados, ações e função objetivo

- **Estado (solução candidata):** uma permutação $\pi = (\pi_1, \ldots, \pi_n)$ de $\mathcal{A}$, que representa a ordem de visita. O espaço de busca é $\mathcal{S} = S_n$, com $|\mathcal{S}| = n!$.
- **Ações:** os movimentos definidos pelos operadores de vizinhança (Seção 2.5). Cada movimento transforma uma permutação em outra.
- **Restrições:** não há restrições rígidas, pois toda permutação é viável. Os prazos d_j são restrições brandas (*soft constraints*), penalizadas na função objetivo. O problema é de otimização, e não de satisfação.
- **Instante de conclusão:** com $\pi_0 = 0$ e $C_{\pi_0} = 0$,

$$C_{\pi_k} = C_{\pi_{k-1}} + t_{\pi_{k-1}\pi_k} + p_{\pi_k}. \qquad (2)$$

- **Função objetivo (*fitness*), a minimizar:** o atraso ponderado total

$$h(\pi) = \sum_{j=1}^{n} w_j \cdot \max(0,\, C_j - d_j). \tag{3}$$

- **Problema:** $\pi^\star = \arg\min_{\pi \in S_n} h(\pi)$.

Na terminologia do algoritmo genético, o **cromossomo** é a permutação completa, o **gene** é uma posição da sequência, o **alelo** é o ativo que a ocupa e a **aptidão** é $h(\pi)$. Como h mede custo, o GA busca o cromossomo de menor aptidão.

2.3 Propriedades úteis

O custo acumulado h é não decrescente ao longo do prefixo da permutação: acrescentar o $(k{+}1)$-ésimo ativo nunca reduz o atraso já acumulado pelos k primeiros. Além disso, o instante C_{π_k} depende apenas do prefixo $(\pi_1, \ldots, \pi_k)$, o que permite avaliar um movimento recalculando somente o sufixo afetado (Seção 3.2).

2.4 Instâncias

Os arquivos `manutencao_*.txt` contêm n, as coordenadas da base e, para cada ativo, $(x_j, y_j, p_j, d_j, w_j)$. A Tabela 1 resume as instâncias e os valores de referência fornecidos no enunciado. O valor da instância pequena é o ótimo global, obtido por busca exata. Os demais são os melhores valores conhecidos, obtidos pelos professores com orçamento muito maior, e servem como alvo, não como meta obrigatória.

Tabela 1: Instâncias e valores de referência de h.

Instância	n	$\sum_j p_j$	Ref. h	Natureza
pequena	12	55	1 875	ótimo global
intermediária	50	239	16 332	melhor conhecido
grande	150	684	60 417	melhor conhecido

2.5 Operadores de vizinhança

Os três operadores clássicos geram sucessores de uma permutação (Figura do enunciado, exemplo com $\pi = (3, 7, 1, 5, 2, 8)$):

- **Swap (troca):** troca os ativos nas posições $i < j$. Há $\binom{n}{2} = \frac{n(n-1)}{2}$ sucessores.
- **Insertion (realocação):** remove o ativo da posição i e o reinsere na posição $j \neq i$. Há $n(n{-}1)$ movimentos, mas a realocação entre posições adjacentes coincide com um *swap* adjacente e é contada duas vezes; o número de sucessores distintos é $(n-1)^2$.
- **2-opt (inversão):** inverte o segmento entre as posições $i < j$. Há $\frac{n(n-1)}{2}$ sucessores, e a inversão de um segmento de tamanho 2 coincide com um *swap* adjacente.

A Tabela 2 apresenta o tamanho de cada vizinhança nas três instâncias. Os operadores não são equivalentes: cada um induz um conjunto distinto de sucessores e, portanto, uma superfície adaptativa diferente para o mesmo problema.

Tabela 2: Tamanho da vizinhança induzida em função de n.

Operador	Fórmula	$n = 12$	$n = 50$	$n = 150$
Swap	$n(n-1)/2$	66	1 225	11 175
Insertion	$(n-1)^2$	121	2 401	22 201
2-opt	$n(n-1)/2$	66	1 225	11 175

Na instância grande, uma varredura completa da vizinhança de *swap* consome 11 175 avaliações, cerca de 3,7% do orçamento de 300 000. Esse custo condiciona o comportamento das variantes de seleção do sucessor (Seção 3.3).

3 Algoritmos

3.1 Arquitetura

Os três algoritmos são métodos da classe `Solver` e acessam o problema apenas pelas classes e funções da Tabela 3. As 9 combinações (3 algoritmos × 3 instâncias) rodam sem mudar o código dos algoritmos, e a vizinhança é escolhida pelo nome do operador (`swap`, `insertion` ou `invert`).

Tabela 3: Interface entre algoritmos e problema no código.

Elemento	Papel
`setOfAssets.shake(rng)`	gera uma permutação aleatória
`objetiveFunction(a,s)`	$h(\pi)$, Eq. (3)
`swap`, `insertion`, `invert`	sucessor de π pelas posições (i, j)
`generatePermutations(n)`	pares (i, j) para varrer a vizinhança
`rng.sample(range(n),2)`	sorteia um sucessor (SA)
`Solver.crossing`	recombinação
`Solver.mutateSolution`	mutação

Organização do código. O código fica em `src/core` (modelo e algoritmos), `src/utils` (leitura da instância e gravação dos resultados) e `main.py`. A classe `Assets` guarda os dados lidos por `readInput` (n, base e, por ativo, posição, p_j, d_j e w_j, com o ativo j no índice $j-1$) e calcula t_{ij} pela Eq. (1). A classe `setOfAssets` é a solução candidata: guarda o vetor π e uma referência ao `Assets` compartilhado, de modo que copiar uma solução copia só π. O construtor rejeita vetores que não sejam permutação de $1, \ldots, n$. Os operadores de vizinhança recebem posições de π $(0, \ldots, n-1)$ e devolvem uma solução nova, sem alterar a atual. A função `objetiveFunction` calcula h pelas Eqs. (2) e (3) em $O(n)$. A classe `Solver` reúne os algoritmos e a tabela `PROTOCOL`, que guarda por instância o orçamento B, o número de execuções e as temperaturas T_0 e T_f do SA. Cada algoritmo identifica a instância pelo n do `Assets` recebido. Em `src/utils/runsComparison.py`, a

função `readHistory` lê o `.csv` de uma execução e recalcula h de cada avaliação, e `compareRuns` usa esses históricos para gerar um gráfico de avaliação × melhor h por quantidade de execuções comparadas (Figura 1).

Linguagem, bibliotecas e aleatoriedade. O código é escrito em Python 3.14.7 com NumPy 2.5.4, usado nos vetores do histórico e nas estatísticas, e Matplotlib 3.11.2, usado nos gráficos; o `requirements.txt` também fixa o pytest 9.1.1. No SA e no *greedy descent*, a única fonte de aleatoriedade de uma execução é um `random.Random` da biblioteca padrão (Mersenne Twister) guardado em `Solver.rng`. Cada algoritmo recria esse gerador a partir de `Solver.seed` ao começar. Por isso, algoritmos diferentes com a mesma semente partem da mesma permutação inicial, e repetir uma execução reproduz o mesmo resultado.

Política de sementes. O `main.py` executa as sementes $0, 1, \ldots, R-1$, com R o número de execuções da Tabela 7, e grava cada execução em `outputs/<instância>/<algoritmo>/seed_<s>.csv`. A semente fica registrada no nome do arquivo. Cada linha do `.csv` é a permutação corrente após uma avaliação, na ordem em que as avaliações ocorreram, e a última linha é a melhor solução da execução. Ao final, o programa imprime o h final de cada semente e a média, o desvio-padrão, o mínimo e a mediana.

Contagem de avaliações. Cada algoritmo conta as avaliações no próprio laço e para ao atingir B. No SA, cada iteração chama h exatamente uma vez, e os vetores do histórico (`Solver.history`) têm B posições, uma por avaliação. A primeira posição corresponde à avaliação da solução inicial.

3.2 Avaliação incremental

Avaliar h do zero custa $O(n)$. Ao aplicar um movimento cuja menor posição afetada é i, o prefixo $(\pi_1, \ldots, \pi_{i-1})$ permanece inalterado, de modo que $C_{\pi_{i-1}}$ e o custo acumulado até $i-1$ podem ser reaproveitados a partir de vetores de prefixo mantidos para a solução corrente. Basta recalcular o sufixo, a um custo $O(n-i+1)$. Isso vale para *swap*, *insertion* e 2-opt: mesmo que os travessias sejam simétricas, os instantes C_j dentro do trecho alterado mudam e precisam ser recalculados. A avaliação incremental não altera a contagem de avaliações, apenas o tempo de parede.

[PREENCHER: vizinhança(s) com avaliação incremental implementada, descrição do teste automático de concordância com a avaliação completa e seu resultado]

O ganho de tempo medido é reportado na Seção **??**.

3.3 (A) *Greedy descent* com *random restart*

A partir de uma permutação aleatória, o algoritmo move-se repetidamente para um sucessor que melhora h, até atingir um **ótimo local**: uma permutação sem sucessor melhor na vizinhança adotada. Ao atingir o ótimo local, aplica-se *random restart*: recomeça-se de nova permutação aleatória, preservando a melhor solução já vista, até esgotar o orçamento B. As variantes de seleção do sucessor são:

- ***Best improvement*:** varre toda a vizinhança e escolhe o melhor sucessor;
- ***First improvement*:** aceita o primeiro sucessor que melhora h;
- **Sucessor aleatório com aceitação de melhora:** sorteia um único sucessor e o aceita somente se melhorar h.

Para o código entregue, foram implementadas ambas as variações de *Best improvement* e *First improvement*.

Sob orçamento fixo, as variantes diferem fortemente em passos de melhoria por avaliação: o *best improvement* paga uma vizinhança inteira por passo, enquanto o *first improvement* e a variante aleatória pagam em média menos. A Seção 5.2 reporta quantos passos de melhoria cada variante deu dentro do orçamento. Para a variante aleatória, o critério de ótimo local não é determinístico; é preciso definir o critério de reinício.

Para detecção de um ótimo local, foi utilzada uma abordagem simples. Caso o algoritmo avance para um sucessor a partir de um determinado estado, o algoritmo identifica que não está em um ótimo local, pois houve melhoria na função objetivo. Entretanto, caso ele não melhore, uma nova solução aleatória é sorteada para que ele inicie o processo de *greedy descent* novamente.

3.4 (B) *Simulated annealing*

Seja π a solução corrente e π' um sucessor sorteado ao acaso. No código, sorteiam-se duas posições distintas com `rng.sample` e aplica-se o *swap*, que é a vizinhança usada pelo SA em todas as instâncias. Se $h(\pi') \le h(\pi)$, aceita-se π'; caso contrário, aceita-se com probabilidade

$$P = \exp\left(-\frac{h(\pi') - h(\pi)}{T}\right), \tag{4}$$

a distribuição de Gibbs. Cada sucessor examinado consome uma avaliação, e a melhor solução vista é guardada à parte da corrente.

Annealing schedule. Adota-se resfriamento geométrico $T \leftarrow \alpha T$. Como cada iteração consome uma avaliação e o orçamento é B, impor $T_f = T_0\,\alpha^{B/m}$ com atualização a cada m iterações fornece

$$\alpha = \left(\frac{T_f}{T_0}\right)^{m/B}. \tag{5}$$

Assim, a temperatura chega a T_f exatamente ao esgotar o orçamento, e o resfriamento é ajustado a B de cada instância. A implementação usa $m = 1$, ou seja, T muda a cada avaliação. A primeira avaliação é a da solução inicial, e o primeiro sucessor ainda é testado com T_0; restam $B-2$ multiplicações até a última avaliação, e por isso o código calcula $\alpha = (T_f/T_0)^{1/(B-2)}$.

Calibração de T_0 e T_f. A escala de T depende da magnitude típica de $\Delta = h(\pi') - h(\pi)$, que vai de dezenas

a dezenas de milhares e muda com a instância. Por isso os parâmetros são calibrados *por instância*, com a própria Eq. (4): dada uma amostra D de valores $\Delta > 0$, a probabilidade média de aceitar um movimento pior à temperatura T é

$$\bar{P}(T) = \frac{1}{|D|} \sum_{\Delta \in D} e^{-\Delta/T}, \tag{6}$$

que cresce com T. Escolhe-se então a temperatura que leva $\bar{P}$ ao alvo desejado. O procedimento foi:

1. para T_0, sortear 2 000 pares (π, π') por instância, com π aleatória e π' obtido por um *swap* aleatório, e guardar os $\Delta > 0$ (cerca de metade dos pares, entre 994 e 1 025 valores);
2. escolher T_0, em número redondo, com $\bar{P}(T_0) \approx 0{,}8$, para que no início a busca aceite cerca de quatro em cada cinco movimentos piores e se comporte quase como *random walk*;
3. para T_f, usar execuções-piloto: em torno da solução final de 5 execuções, sortear 1 000 *swaps* de cada uma (5 000 movimentos) e escolher T_f com $\bar{P}(T_f)$ entre 1% e 2,5%, para que o fim da execução se comporte quase como *greedy descent*;
4. conferir a escolha pelo gráfico da taxa de aceitação ao longo da execução (Figura 5).

A amostra de T_f é tirada em torno das soluções finais porque ali os Δ têm outra escala: na instância grande, a mediana de $\Delta > 0$ é 7 688 em permutações aleatórias e 18 564 em torno das soluções finais. A Tabela 4 resume as amostras, e a Tabela 5 traz os valores adotados.

Tabela 4: Amostras de calibração do SA ($\Delta > 0$ de movimentos *swap*) e aceitação média estimada pela Eq. (6) nas temperaturas adotadas.

	π aleatória		π final (piloto)	
Instância	$\bar{\Delta}$	$\bar{P}(T_0)$	mediana Δ	$\bar{P}(T_f)$
pequena	788	0,79	1 214	0,013
intermediária	4 245	0,75	7 322	0,016
grande	10 346	0,80	18 564	0,025

Tabela 5: Parâmetros do *simulated annealing* por instância ($m = 1$).

Instância	T_0	T_f	α	Vizinhança
pequena	3 000	25	0,999904	*swap*
intermediária	13 000	180	0,999971	*swap*
grande	41 000	650	0,999986	*swap*

Taxa de aceitação. Registra-se, em janelas de avaliações, a fração de movimentos *piores* aceitos. Uma têmpera bem calibrada começa quase como *random walk* e termina quase como *greedy descent*; um gráfico que permanece alto indica T alta demais, e um que cai a zero logo no início indica T baixa demais. No código, o histórico de cada execução guarda, por avaliação, a temperatura, o h do sucessor, o h corrente, o melhor h e se o sucessor era pior e se foi aceito. A Figura 5 divide o orçamento em 50 janelas e mostra a média das execuções. A taxa começa em 0,77, 0,73 e 0,77 e termina em 0,008, 0,020 e 0,025 nas instâncias pequena, intermediária e grande, perto dos alvos da calibração.

3.5 (C) Algoritmo genético

O GA mantém uma população de N cromossomos e repete: seleção, cruzamento com probabilidade P_c, mutação com probabilidade P_m e elitismo, até esgotar o orçamento de avaliações. A população inicial é uma amostra aleatória de N permutações.

Seleção e conversão custo→aptidão. Como h é de minimização, o operador de seleção deve favorecer valores *menores*. No torneio de tamanho k, basta escolher o indivíduo de menor h entre k sorteados, sem conversão. Na roleta, é necessária uma aptidão $f(\pi) \geq 0$ crescente com a qualidade, por exemplo $f(\pi) = h_{\max} - h(\pi)$ (com $h_{\max}$ o pior valor da geração) ou $f(\pi) = 1/(1 + h(\pi))$.

[PREENCHER: operador de seleção adotado, tamanho do torneio ou fórmula de conversão custo→aptidão efetivamente usada, e justificativa]

Cruzamento. Cruzamentos clássicos de um ou múltiplos pontos duplicariam ativos e gerariam cromossomos inválidos; é necessário um operador que preserve a permutação (OX, PMX ou cruzamento por arestas).

[PREENCHER: operador de cruzamento adotado (OX, PMX ou arestas) e justificativa]

A título de ilustração, o *Order Crossover* (OX) opera como segue, com $n = 8$, pais $P_1 = (1\,2\,3\,4\,5\,6\,7\,8)$ e $P_2 = (3\,7\,5\,1\,6\,8\,2\,4)$ e pontos de corte antes da posição 4 e depois da posição 6:

1. copia-se o segmento de P_1 entre os cortes para o filho: $(___\,4\,5\,6\,__)$;
2. percorre-se P_2 a partir da posição seguinte ao segundo corte (posição 7), de forma circular: $2, 4, 3, 7, 5, 1, 6, 8$;
3. descartam-se os ativos já presentes no segmento $(4, 5, 6)$, restando $2, 3, 7, 1, 8$;
4. preenchem-se as posições livres a partir da posição 7, também circularmente: posições $7, 8, 1, 2, 3$ recebem $2, 3, 7, 1, 8$;
5. o filho é $(7\,1\,8\,4\,5\,6\,2\,3)$, que é uma permutação válida.

Mutação. Aplica-se uma pequena taxa de mutação para inserir diversidade e favorecer a exploração. O ponto de partida é a *swap mutation*; os demais operadores da vizinhança também são admissíveis.

[PREENCHER: operador de mutação adotado]

Elitismo e parâmetros. O elitismo preserva os e melhores indivíduos de cada geração. A Tabela 6 reúne os parâmetros. Cada indivíduo novo avaliado consome uma avaliação, e o algoritmo para exatamente em B.

Tabela 6: Parâmetros do algoritmo genético.

Parâmetro	pequena	interm.	grande
N (população)	[PREENCHER]	[PREENCHER]	[PREENCHER]
P_c	[PREENCHER]	[PREENCHER]	[PREENCHER]
P_m	[PREENCHER]	[PREENCHER]	[PREENCHER]
Elitismo (e)	[PREENCHER]	[PREENCHER]	[PREENCHER]
Tamanho do torneio	[PREENCHER]	[PREENCHER]	[PREENCHER]

[PREENCHER: como os parâmetros foram escolhidos e testados]

Diversidade. A cada geração registra-se o número de cromossomos distintos e a distância média entre pares de indivíduos, definida pela distância de Hamming entre posições $d(\pi,\sigma)=|\{k:\pi_k\neq\sigma_k\}|$. A queda dessas medidas diagnostica a convergência prematura.

4 Protocolo Experimental

4.1 Orçamento e execuções

Todos os algoritmos gastam o mesmo recurso: o número de avaliações de h. Cada sucessor examinado pela busca local conta 1, e cada indivíduo avaliado pelo GA conta 1. Os algoritmos param exatamente ao atingir o orçamento (Tabela 7).

Tabela 7: Orçamento e execuções independentes.

Instância	Orçamento B	Execuções
pequena ($n=12$)	50 000	30
intermediária ($n=50$)	150 000	30
grande ($n=150$)	300 000	10

4.2 Métricas e estatística

Para cada combinação algoritmo × instância, registra-se o melhor h de cada execução e calculam-se média, desvio-padrão, mínimo e mediana. O gap relativo ao ótimo na instância pequena é $\text{gap}=(h-h^\star)/h^\star$, com $h^\star=1875$. Registram-se também o tempo de parede por execução e o número de avaliações.

As comparações par a par usam o teste não paramétrico de Mann–Whitney [5], bilateral, com nível de significância $\alpha_s=0{,}05$. Como são três comparações por instância, sugere-se correção de Holm.

[PREENCHER: teste e correção para múltiplas comparações efetivamente usados e a biblioteca empregada]

Médias próximas com dispersões grandes não sustentam conclusão; as afirmações de superioridade no texto vêm acompanhadas do p-valor.

4.3 Curvas de convergência

Para cada instância, registra-se o melhor h encontrado em função do número de avaliações gastas, em uma grade fixa de pontos, e plota-se a mediana das execuções por algoritmo. Para o GA, acrescenta-se a curva da média da população; a distância entre as duas curvas indica a perda de diversidade.

4.4 Ambiente de execução

As execuções do SA rodaram em um Intel Core i5-14600KF (20 *threads*) com 15 GiB de RAM, Arch Linux (kernel 7.1.9), Python 3.14.7 e NumPy 2.5.4. As execuções são sequenciais e usam uma única *thread*. O tempo é medido com `time.perf_counter` dentro do algoritmo e não inclui a leitura da instância nem a gravação do `.csv`.

[PREENCHER: ambiente das execuções do *greedy descent* e do GA, se diferente]

4.5 Reprodutibilidade

O código acompanha um `README` com os comandos exatos para reproduzir cada tabela e figura. Os resultados brutos de todas as execuções são gravados em arquivos `.csv`.

O código está em `https://github.com/Lucas-Szuster/FUNDIA-TP2`. Cada combinação algoritmo × instância é executada por `python main.py <instância> <algoritmo>`, com `<instância>` em {`small`, `medium`, `big`} e `<algoritmo>` em {`greedyDescent`, `simulatedAnnealing`, `geneticAlgorithm`}. Os resultados do SA na Tabela 10 vêm de `python main.py big simulatedAnnealing` e dos comandos equivalentes para `small` e `medium`. A opção `--runs` muda o número de execuções do protocolo, e `python -m src.utils.runsComparison <instância> <algoritmo>` gera os gráficos de comparação entre 5, 10 e 15 execuções em `outputs/<instância>/<algoritmo>/`.

[PREENCHER: comandos do README para as demais tabelas e figuras (ver Anexo A)]

5 Resultados e Análise

5.1 T1: validação da função de avaliação

A função h foi validada com os dois valores do enunciado para `manutencao_pequena.txt`: a permutação identidade $\pi=(1,\dots,12)$ deve resultar em $h(\pi)=4366$, e o ótimo global conhecido é $h^\star=1875$. Uma heurística gulosa razoável (menor acréscimo imediato de atraso) alcança 3168, cerca de 69% acima do ótimo.

Tabela 8: Saída do programa nos casos de verificação.

Caso	Esperado	Obtido
Identidade, instância pequena	4 366	[PREENCHER]
Ótimo global, instância pequena	1 875	[PREENCHER]

[PREENCHER: saída literal do programa para os dois casos; opcionalmente o exemplo com $n=4$ (custos 39 e 22)]

5.2 T2: comparação das vizinhanças

Usando *greedy descent* com *random restart* e o mesmo orçamento, comparam-se *swap*, *insertion* e 2-opt nas três instâncias.

Tabela 9: Comparação das vizinhanças no *greedy descent* com *random restart* (média ± desvio-padrão).

Instância	Vizinhança	Melhor h (média ± dp)	Passos de melhoria	Reinícios	Tempo por movimento (s)
pequena	Swap	1875.0 ± 0.0	1722.2 ± 28.8	88.4 ± 2.8	2.45×10^{-5}
pequena	Insertion	1947.4 ± 97.2	1509.4 ± 33.4	91.2 ± 2.6	3.02×10^{-5}
pequena	2-opt	1875.0 ± 0.0	1811.0 ± 33.8	92.0 ± 2.4	3.36×10^{-5}
intermediária	Swap	28575.4 ± 3318.2	347.6 ± 37.1	1.2 ± 0.4	7.44×10^{-5}
intermediária	Insertion	42434.3 ± 5139.2	290.6 ± 33.5	1.4 ± 0.5	1.13×10^{-4}
intermediária	2-opt	20323.7 ± 1323.4	310.3 ± 22.1	1.1 ± 0.3	7.52×10^{-5}
grande	Swap	385302.5 ± 47298.2	412.9 ± 34.2	0.0 ± 0.0	1.88×10^{-4}
grande	Insertion	708284.8 ± 44363.6	240.9 ± 29.4	0.0 ± 0.0	1.94×10^{-4}
grande	2-opt	522030.8 ± 34483.7	254.8 ± 30.2	0.0 ± 0.0	1.58×10^{-4}

Os resultados mostram que o desempenho das vizinhanças depende do tamanho da instância. Na instância pequena, *swap* e *2-opt* alcançaram o mesmo valor médio de $h = 1875,0$, sem variação entre as execuções. Entretanto, *swap* apresentou menor tempo por movimento. Já *insertion* obteve uma média superior de $h = 1947,4$, indicando soluções de menor qualidade nesse cenário.

Na instância intermediária, *2-opt* apresentou o melhor resultado médio, com $h = 20323,7$, além de menor desvio-padrão entre as três vizinhanças.

Na instância grande, por outro lado, *swap* obteve a menor média de $h = 385302,5$, enquanto *2-opt* apresentou o menor tempo por movimento. A vizinhança *insertion* apresentou os piores resultados médios nas três instâncias, sem vantagem de tempo que compensasse essa diferença.

O número de passos de melhoria deve ser interpretado em conjunto com a qualidade das soluções e o custo das avaliações, pois uma quantidade maior de passos não implica necessariamente um resultado melhor. Na instância grande, uma varredura completa da vizinhança *swap* exige 11175 avaliações, o que limita a exploração sob o orçamento de 300000 avaliações. Como a estratégia de primeira melhoria interrompe a varredura ao encontrar um candidato melhor, o número de avaliações efetivamente utilizado por passo pode ser menor.

Dessa forma, *2-opt* destaca-se na instância intermediária e apresenta menor custo por movimento na instância grande, enquanto *swap* é preferível quando a prioridade é a qualidade da solução na instância grande. A escolha final deve considerar o equilíbrio entre qualidade e custo computacional. Ressalta-se, ainda, que os tempos apresentados refletem a implementação utilizada, não sendo suficientes para determinar qual vizinhança seria mais eficiente com avaliação incremental da função objetivo.

5.3 T3: resultados principais

[PREENCHER: valores do *greedy descent* e do GA na Tabela 10 a partir dos resultados em disco]

Simulated annealing. O gap do SA na Tabela 10 é o da média; o tempo é o do algoritmo, sem a gravação do `.csv`. Rodar de novo as sementes das instâncias pequena e intermediária reproduz os mesmos h finais gravados nos `.csv`.

Na instância pequena, o SA chegou a $h^\star = 1875$ em 27 das 30 execuções. As outras três terminaram em 1 887, 1 937 e 1 993. Na mediana, a melhor solução apareceu na avaliação 31 104 (62% do orçamento), e o resto da execução ficou perto do ótimo.

Na intermediária, a média ficou 33% acima do melhor valor conhecido (16 332), e a melhor execução (18 612), 14% acima. Na grande, a média foi 2,7 vezes o melhor conhecido (60 417), e mesmo a melhor execução (137 865) ficou 128% acima. Nessas duas instâncias a melhor solução só aparece no fim: na mediana, na avaliação 147 038 de 150 000 (98%) e 298 680 de 300 000 (99,6%). Na grande, a mediana do melhor h ainda caiu de 188 806 para 161 576 nos últimos 10% do orçamento, ou seja, o orçamento acaba com a busca ainda descendo.

A razão entre o orçamento e o tamanho da vizinhança de *swap*, $B/|N|$, ajuda a explicar a piora com n: ela vale 758 na pequena (50 000/66), 122 na intermediária e 27 na grande. Como T_0 foi calibrada em permutações aleatórias e o resfriamento é geométrico, até a metade do orçamento o SA ainda aceita mais de 15% dos movimentos piores nas três instâncias (Figura 5). Na grande, a mediana do h corrente na janela que começa na metade da execução é 605 532, dez vezes o valor de referência, e restam poucas avaliações em temperatura baixa para refinar a solução. Uma T_0 menor ou um orçamento maior deslocariam a fase de refinamento para mais cedo ou a tornariam mais longa; esse ajuste não foi testado neste trabalho.

Convergência das execuções do SA. A Figura 1 mostra o melhor h de cada execução ao longo do orçamento, com as 5, 10 e 15 primeiras sementes de cada instância, e a Tabela 11 compara entre si todas as execuções do protocolo (30, 30 e 10). Para comparar as soluções finais, usam-se a distância de Hamming da Seção 3.5 e o deslocamento médio de posição, $\frac{1}{n}\sum_j |\mathrm{pos}_\pi(j) - \mathrm{pos}_\sigma(j)|$, em que $\mathrm{pos}_\pi(j)$ é a posição do ativo j em π. Entre duas permutações sorteadas ao acaso, os valores esperados são $n-1$ e $(n^2-1)/(3n)$.

Na instância pequena, as execuções convergem para a mesma solução. As 27 que chegam a $h^\star = 1875$ terminam na mesma permutação, $(5,4,1,6,7,2,8,12,3,10,9,11)$, a partir de 27 permutações iniciais diferentes. Entre as 30 execuções há só 4 soluções finais distintas, e a distância de Hamming média entre pares é 0,85. O ótimo é alcançado entre as avaliações 24 158 e 36 987 (48% a 74% do orçamento). A partir de 75% do orçamento, o primeiro quartil, a mediana e o terceiro quartil do melhor h valem 1 875.

Tabela 10: Resultados dos três algoritmos (melhor h por execução). O gap é calculated apenas na instância pequena, com $h^\star = 1875$.

Instância	Algoritmo	Média ± dp	Mediana	Mínimo	Gap	Tempo méd
pequena	Greedy + restart	1875,0 ± 0,0	1875	1875	0,00%	1,24
pequena	Sim. annealing	1 881,4 ± 24,0	1 875	1 875	0,34%	0,39
pequena	Alg. genético	[PREENCHER]	[PREENCHER]	[PREENCHER]	[PREENCHER]	[PREENC
intermediária	Greedy + restart	28575,4 ± 3318,2	28736	21887	—	11,23
intermediária	Sim. annealing	21 739,5 ± 1 499,1	21 609,5	18 612	—	2,38
intermediária	Alg. genético	[PREENCHER]	[PREENCHER]	[PREENCHER]	—	[PREENC
grande	Greedy + restart	385302,5 ± 47298,2	391516	298989	—	55,97
grande	Sim. annealing	161 037,2 ± 10 750,9	161 576	137 865	—	10,76
grande	Alg. genético	[PREENCHER]	[PREENCHER]	[PREENCHER]	—	[PREENC

Na intermediária, as 30 execuções terminam em 30 soluções diferentes, com distância de Hamming média de 47,0, perto das 49 esperadas entre permutações aleatórias. O deslocamento médio de posição, 9,4, fica abaixo dos 16,7 esperados ao acaso. Os ativos tendem a ocupar a mesma região da sequência em todas as execuções, mas quase nenhum fica na mesma posição. A grande repete o padrão, com Hamming médio de 148,1 (149 ao acaso) e deslocamento de 33,4 (50,0 ao acaso).

O coeficiente de variação (CV) do melhor h entre execuções evolui de forma diferente em cada instância. Na pequena, ele cai de 8,1% com 25% do orçamento para 1,3% no fim. Na grande, ele sobe de 2,2% para 6,7%: no início todas as execuções estão na mesma faixa de h alto, e as diferenças aparecem quando elas descem em ritmos diferentes. Na intermediária, o CV fica entre 4,9% e 8,7% durante toda a execução. Na mediana, uma execução chega a 5% do seu próprio valor final com 56% do orçamento na pequena, 93% na intermediária e 98% na grande. Nas duas maiores, as curvas da Figura 1 ainda descem quando o orçamento acaba, e cada execução para em um ponto diferente dessa descida. Por isso as soluções finais diferem tanto entre si.

Tabela 11: Convergência das execuções do SA em cada instância: soluções finais distintas, distância de Hamming e deslocamento médio de posição entre soluções finais, coeficiente de variação (CV) do melhor h com 25%, 50% e 100% do orçamento e fração mediana do orçamento até chegar a 5% do h final. Entre parênteses, o valor esperado entre duas permutações aleatórias.

Métrica	pequena	interm.	grande
Finais distintas	4/30	30/30	10/10
Hamming	0,85 (11)	47,0 (49)	148,1 (149)
Deslocamento	0,12 (3,97)	9,4 (16,7)	33,4 (50,0)
CV 25% (%)	8,1	4,9	2,2
CV 50% (%)	7,0	6,4	3,2
CV 100% (%)	1,3	6,9	6,7
Até 5% do final	0,56	0,93	0,98

Quantidade de execuções. Os gráficos da Figura 1 usam as sementes 0 a $k-1$ para $k = 5$, 10 e 15. Na grande, o protocolo prevê 10 execuções; as sementes 10 a 14 foram rodadas só para esta comparação, com `python main.py big simulatedAnnealing --runs 15`, e não entram nas Tabelas 10 e 11. A mediana das curvas quase não muda com k. Com metade do orçamento, ela vale 2 107, 2 052 e 2 107 na pequena, 49 855, 50 185 e 52 009 na intermediária e 522 533, 523 435 e 524 337 na grande. A média do h final varia no máximo 2% entre $k = 5$ e $k = 15$ (Tabela 12), então 5 execuções já mostram a forma da descida e o nível de h que o SA alcança.

A dispersão depende bem mais de k. Na grande, o desvio-padrão do h final passa de 5 019 com 5 execuções para 10 751 com 10 e 12 270 com 15, porque a melhor execução (semente 5, 137 865) e a pior (semente 13, 189 046) estão fora das 5 primeiras. Na intermediária, a melhor execução (semente 7, 18 612) só aparece a partir de $k = 10$. Na pequena acontece o contrário: a única das 15 primeiras execuções que não chega a 1 875 (semente 3, 1 937) já está entre as 5 primeiras, e o desvio-padrão cai de 27,7 para 16,0 à medida que entram execuções que terminam no ótimo. Cinco execuções bastam para descrever a curva típica do SA, mas subestimam a variabilidade: na grande, o desvio-padrão medido com 5 execuções é menos da metade do medido com 10 ou 15. Isso afeta os testes de Mann–Whitney da Tabela 13, que dependem da dispersão das amostras.

Tabela 12: h final do SA com as k primeiras sementes de cada instância.

Instância	k	Média ± dp	Mínimo	Máximo
pequena	5	1 887,4 ± 27,7	1 875	1 937
	10	1 881,2 ± 19,6	1 875	1 937
	15	1 879,1 ± 16,0	1 875	1 937
interm.	5	21 517,4 ± 1 776,6	19 872	23 622
	10	21 162,4 ± 1 711,7	18 612	23 622
	15	21 276,1 ± 1 526,8	18 612	23 622
grande	5	160 267,4 ± 5 018,7	153 192	166 807
	10	161 037,2 ± 10 750,9	137 865	180 136
	15	163 411,2 ± 12 269,6	137 865	189 046

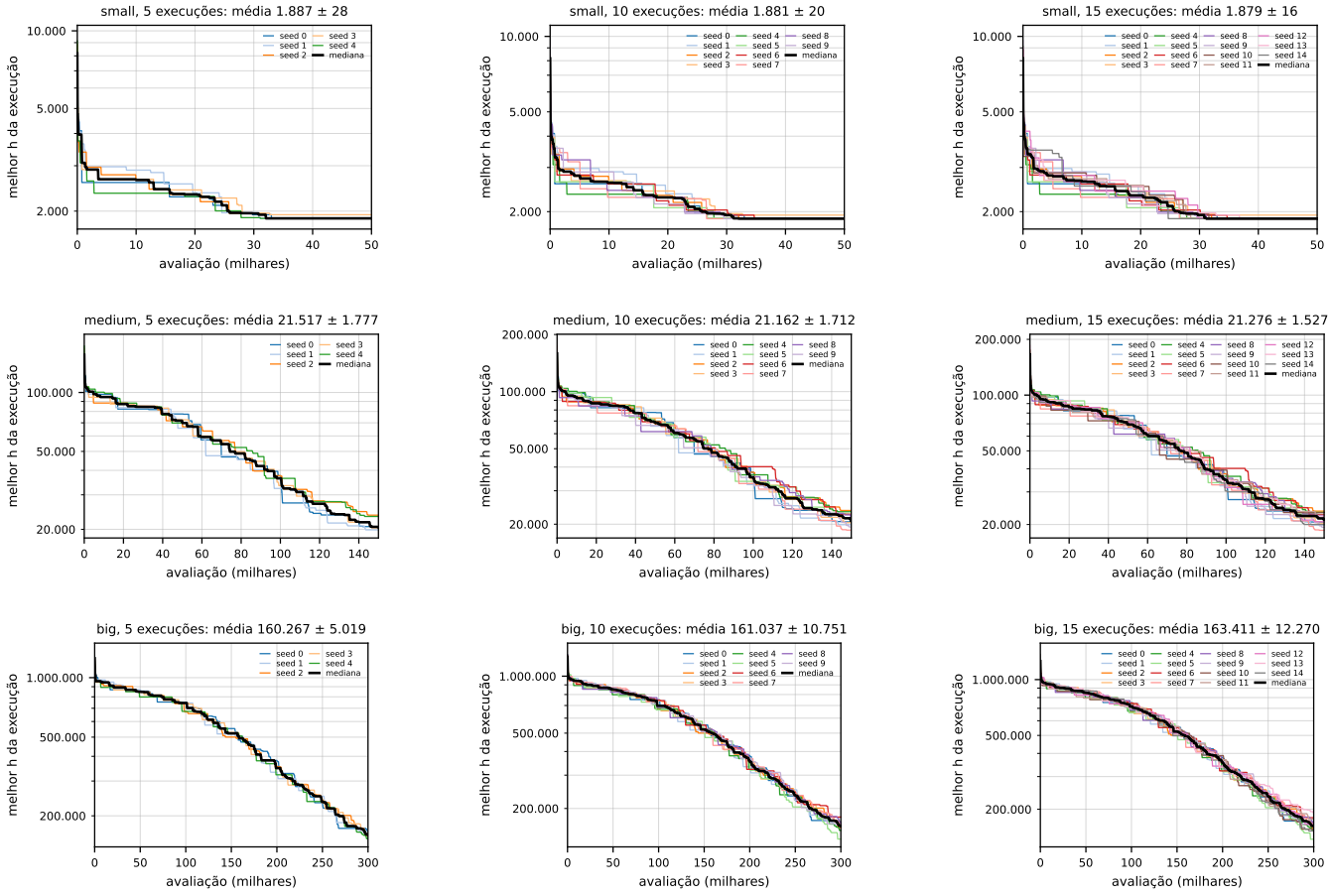


Figura 1: Avaliação × melhor h de cada execução do SA, gráficos gerados por `compareRuns`. Linhas: instâncias pequena (`small`), intermediária (`medium`) e grande (`big`). Colunas: sementes 0 a 4, 0 a 9 e 0 a 14. O eixo y está em escala logarítmica, a linha preta é a mediana das execuções do gráfico, e o título traz a média ± desvio-padrão do h final.

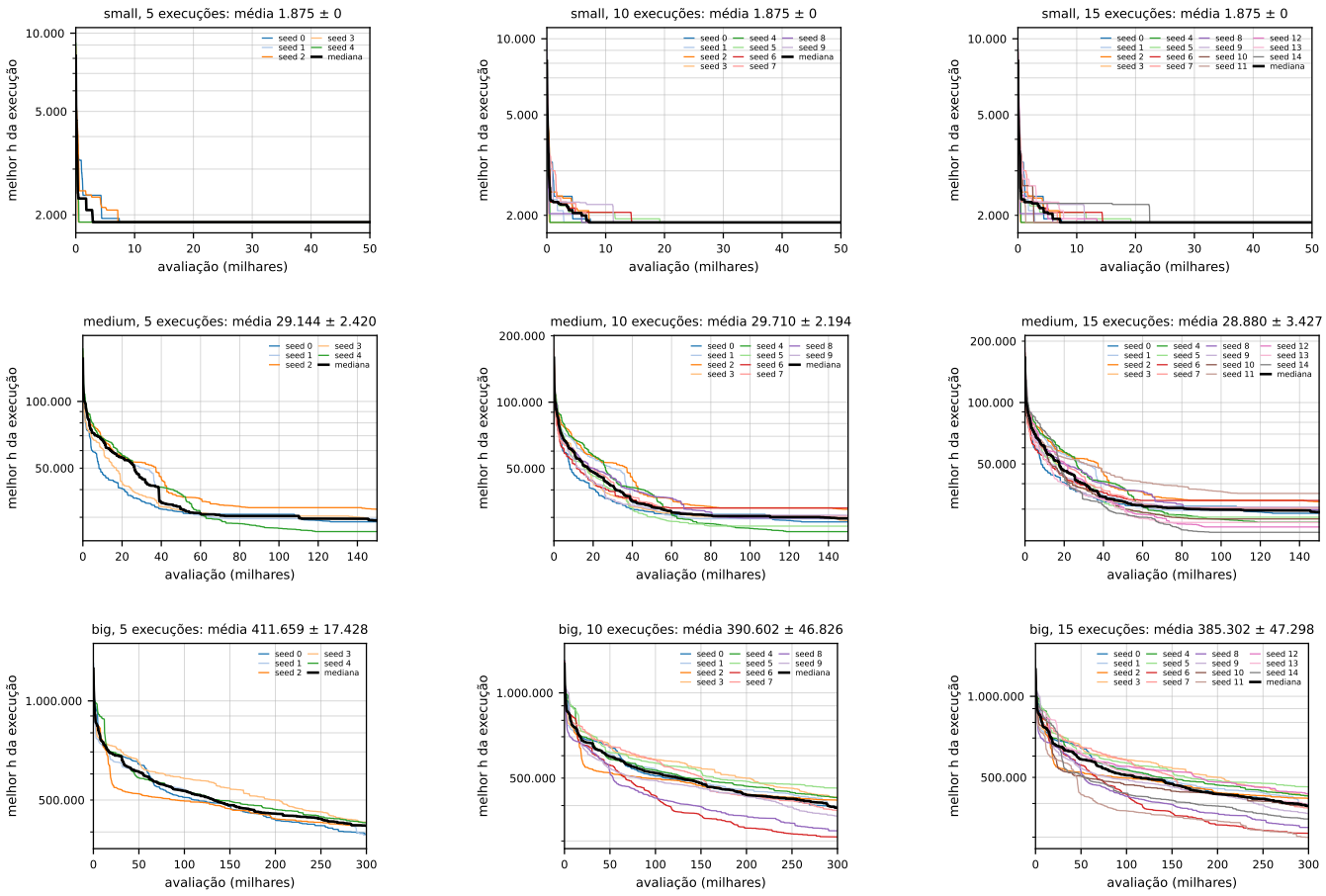


Figura 2: Avaliação × melhor h de cada execução do GD, com método *swap* gráficos gerados por `compareRuns`. Linhas: instâncias pequena (`small`), intermediária (`medium`) e grande (`big`). Colunas: sementes 0 a 4, 0 a 9 e 0 a 14. O eixo y está em escala logarítmica, a linha preta é a mediana das execuções do gráfico, e o título traz a média ± desvio-padrão do h final.

Boxplots. A Figura **??** compara os três algoritmos em cada instância.

Curvas de convergência. A Figura 4 apresenta a mediana do melhor h em função das avaliações, e, para o GA, a curva da média da população.

Testes estatísticos.

Tabela 13: p-valores do teste de Mann–Whitney (bilateral).

Comparação	pequena	interm.	grande
Greedy vs. SA	[PREENCHER]	[PREENCHER]	[PREENCHER]
Greedy vs. GA	[PREENCHER]	[PREENCHER]	[PREENCHER]
SA vs. GA	[PREENCHER]	[PREENCHER]	[PREENCHER]

[PREENCHER: p-valores (e correção, se aplicável) e leitura de quais diferenças são estatisticamente significativas]

Tempo e avaliações. A Tabela 10 apresenta o tempo médio por execução ao lado do número de avaliações. O orçamento é igual entre algoritmos, mas o custo por avaliação não é: operadores como o cruzamento, a avaliação incremental e a manutenção de estruturas da população fazem o tempo de parede diferir mesmo com avaliações idênticas.

[PREENCHER: discussão da diferença entre as métricas (avaliações vs. tempo de parede) com base nos tempos medidos]

5.4 Discussão

Ótimos locais. O SA atingiu o ótimo global $h^\star = 1875$ em 27 das 30 execuções da instância pequena. **[PREENCHER: número de ótimos locais distintos do *greedy descent* na instância pequena; em quantas execuções cada algoritmo atingiu $h^\star = 1875$; relação com a discussão de ótimo local × ótimo global]**

Exploração × intensificação. Em cada algoritmo, identificam-se os seguintes mecanismos candidatos; a análise deve apoiá-los em dados (Seção 4):

- *Greedy + restart*: a descida intensifica; o *restart* explora.
- *Simulated annealing*: a aceitação de piores (alta T) explora; o resfriamento leva à intensificação.
- GA: o cruzamento e a mutação exploram; a seleção e o elitismo intensificam.

No SA, a taxa de aceitação de movimentos piores (Figura 5) cai de cerca de 0,75 para 0,01 a 0,03 nas três instâncias, sem ficar alta até o fim e sem zerar logo no início. Na instância pequena a exploração termina com tempo de sobra, e 27 das 30 execuções chegam ao ótimo. Nas maiores, a mesma forma de curva deixa pouco orçamento para intensificar: a metade inicial do orçamento é gasta em uma região de h muito alto (na grande, a mediana do h corrente nas primeiras janelas passa de 10^6), e a melhor solução ainda melhora na última janela.

[PREENCHER: evidências nos dados de onde o equilíbrio foi bem ou mal ajustado: diversidade do GA e número de reinícios do *greedy*]

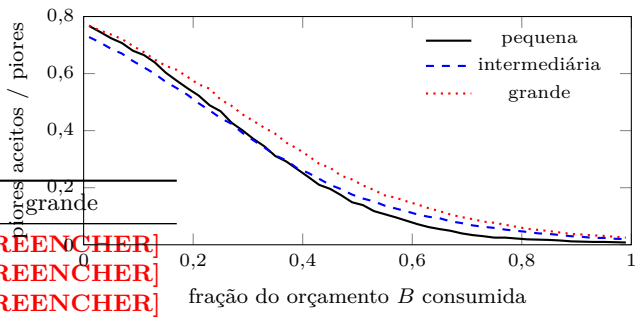


Figura 5: Taxa de aceitação de movimentos piores no *simulated annealing*: fração dos sucessores piores que foram aceitos em cada uma de 50 janelas do orçamento, média das execuções (30, 30 e 10).

[FIGURA PENDENTE]
diversidade da população do GA por geração (arquivo fig_diversidade.pdf)

Figura 6: Evolução da diversidade da população do algoritmo genético.

Efeito da escala. **[PREENCHER: o ranking muda entre a instância pequena e a grande? Hipótese (se mudar) ou evidência estatística de estabilidade (se não mudar)]**

Quando o algoritmo genético compensa? [PREENCHER: existe instância em que o GA supera significativamente as duas abordagens de busca local? Citar os p-valores]

5.4.1 Experimento adicional: a recombinação combina bons trechos?

Hipótese. O GA só traz vantagem sobre a busca local se a recombinação conseguir juntar bons trechos de sequência de dois pais em um filho melhor, isto é, se a codificação por permutação preservar blocos úteis (por exemplo, subsequências de ativos com prazos próximos visitadas em boa ordem). Se o cruzamento produzir filhos em geral piores que ambos os pais, o GA funciona como uma busca aleatória guiada pela mutação e pela seleção.

Experimento proposto. Duas medidas complementares sobre o mesmo orçamento e as mesmas instâncias:

1. **Ablação:** comparar o GA completo com uma versão sem cruzamento ($P_c = 0$, apenas mutação, seleção e elitismo), com o mesmo N e B, usando Mann–Whitney sobre o melhor h final.

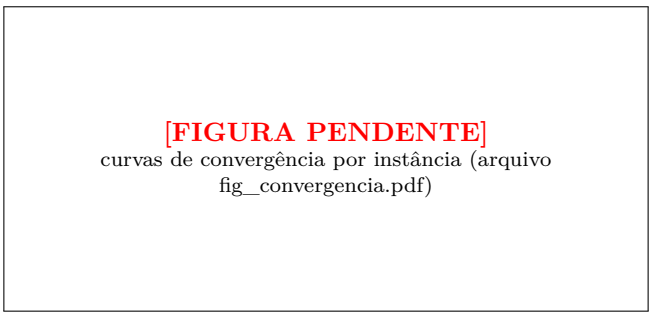


Figura 4: Curvas de convergência (mediana das execuções). A distância entre a curva do melhor e a da média da população do GA indica a perda de diversidade.

2. **Qualidade dos filhos:** em cada geração, registrar a fração de filhos com h menor que o de ambos os pais, menor que o do melhor pai e maior que o de ambos, e como essas frações evoluem à medida que a diversidade cai.

Tabela 14: Experimento adicional: efeito do cruzamento.

Métrica	pequena	interm.	grande
Melhor h, GA completo (média)	[PREENCHER]	[PREENCHER]	[PREENCHER]
Melhor h, GA sem cruzamento (média)	[PREENCHER]	[PREENCHER]	[PREENCHER]
p-valor (Mann–Whitney)	[PREENCHER]	[PREENCHER]	[PREENCHER]
Filhos melhores que ambos os pais (%)	[PREENCHER]	[PREENCHER]	[PREENCHER]
Filhos piores que ambos os pais (%)	[PREENCHER]	[PREENCHER]	[PREENCHER]

[PREENCHER: resultados do experimento e a interpretação: a hipótese foi sustentada ou refutada pelos dados?]

6 Conclusões

[PREENCHER: síntese dos resultados, lições sobre exploração e intensificação, conclusão sobre a recombinação, limitações e trabalhos futuros]

Referências

[1] D. Poole and A. Mackworth, *Artificial Intelligence: Foundations of Computational Agents*, 3rd ed. Cambridge University Press, 2023. Cap. 4.

[2] A. E. Eiben and J. E. Smith, *Introduction to Evolutionary Computing*. Springer-Verlag, 2003.

[3] A. Gaspar-Cunha, R. H. C. Takahashi, and C. H. Antunes (Orgs.), *Manual de Computação Evolutiva e Metaheurística*. Editora UFMG, 2013.

[4] S. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*, 3rd ed. Pearson, 2016. Cap. 4.

[5] H. B. Mann and D. R. Whitney, "On a test of whether one of two random variables is stochastically larger than the other," *Annals of Mathematical Statistics*, vol. 18, no. 1, pp. 50–60, 1947.

A Reprodução dos resultados

Tabela 15: Mapa de reprodução (tabelas e figuras → comandos).

Item	Comando
Tabela 8 (T1)	[PREENCHER]
Tabela 9 (T2)	[PREENCHER]
Tabela ??	[PREENCHER]
Tabela 10	`python main.py <instância> <algoritmo>` uma vez por combinação
Tabela 11	[PREENCHER]
Tabela 13	[PREENCHER]
Tabela 14	[PREENCHER]
Figura ??	[PREENCHER]
Figura 4	[PREENCHER]
Figura 5	[PREENCHER]
Figura 1, Tabela 12	`python main.py big simulatedAnnealing --runs 15` e `python -m src.utils.runsComparison <instância> simulatedAnnealing`
Figura 6	[PREENCHER]